

MERN Stack Internship

DAY 7 ASSIGNMENT REPORT

Name: NIKHIL MANDLOI
Course: BTech CSIT 1 year
Position : MERNStackINTERN
Allocated Date: 7/06/26
Submission Date: 7/006/26

MERN Stack Internship – Day 7 Report

Assignment Overview

This report contains the complete submission for Day 7 of the MERN Stack Internship. The tasks focused on standardizing global state management across team projects using React Context API and integrating ESLint into the React workspace to enforce consistent code-quality standards and formatting guidelines.

Task 7.1 – State Framework Standardization

Objective

The objective of this task was to standardize application-wide state management by implementing React Context API. This approach ensures that all team members use a common global state structure, reducing inconsistencies and improving scalability for larger projects.

Concepts Learned

React Context API

React Context API is a built-in feature that allows data to be shared globally between components without passing props manually through multiple levels.

Benefits:

- Centralized state management
 - Reduced prop drilling
 - Better code organization
 - Improved scalability
-

Global State

Global state refers to data that can be accessed by multiple components throughout the application.

Example:

Student data fetched from the backend can be accessed by:

- App Component
 - StudentCard Component
 - Future Dashboard Components
 - Future Profile Components
-

Context Provider

A Provider is used to make data available to all child components.

Example:

```
<StudentContext.Provider value={{ students }}>
  {children}
</StudentContext.Provider>
```

Implementation

A new Context folder was created inside the React project.

Folder Structure:

```
src/
|
|—— components/
|   └── StudentCard.jsx
|
|—— context/
|   └── StudentContext.jsx
|
|—— App.jsx
└── main.jsx
```

Student data fetching logic was moved from App.jsx into StudentContext.jsx.

The StudentProvider component was wrapped around App inside main.jsx.

The useContext Hook was used inside App.jsx to access student data globally.

State Flow

MongoDB

↓

Express API

↓

StudentContext

↓

React Context Provider

↓

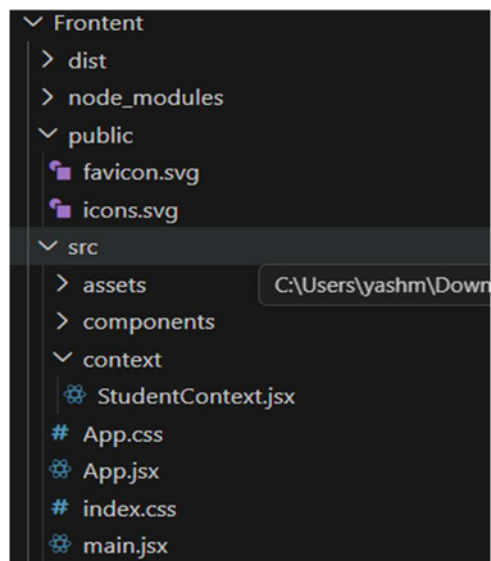
App Component

↓

StudentCard Component

↓

Browser UI



```

import { createContext, useState, useEffect } from "react";

export const StudentContext = createContext();

function StudentProvider({ children }) {

  const [students, setStudents] = useState([]);

  useEffect(() => {

    fetch(`${import.meta.env.VITE_API_URL}/students`)
      .then((response) => response.json())
      .then((data) => {

        setStudents(data);

      })
      .catch((error) => {

        console.log(error);

      });

  }, []);

  return (

    <StudentContext.Provider value={{ students }}>

      {children}

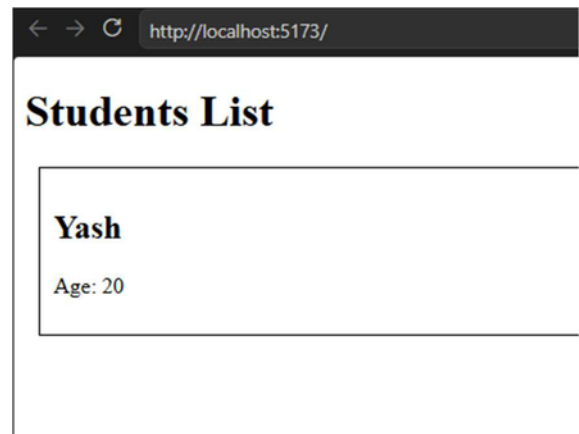
    </StudentContext.Provider>

  );

}

export default StudentProvider;

```



Task 7.1 Source Code

StudentContext.jsx

```
import { createContext, useState, useEffect } from "react";
```

```
export const StudentContext = createContext();
```

```
function StudentProvider({ children }) {
```

```
  const [students, setStudents] = useState([]);
```

```
  useEffect(() => {
```

```
    fetch(`${import.meta.env.VITE_API_URL}/students`)
```

```
      .then((response) => response.json())
```

```
      .then((data) => {
```

```
        setStudents(data);

    });

}, []);

return(

    <StudentContext.Provider value={{ students }}>

        {children}

    </StudentContext.Provider>

);

}
```

exportdefault StudentProvider;

Task 7.2 – Workspace Linting Integration

Objective

The objective of this task was to integrate ESLint into the React workspace to automatically detect coding issues, enforce standardized formatting rules, and maintain code quality across all future project contributions.

Concepts Learned

ESLint

ESLint is a static code analysis tool used to identify potential problems in JavaScript and React applications.

Benefits:

- Detects coding mistakes
 - Enforces coding standards
 - Improves readability
 - Maintains team consistency
-

Linting

Linting is the process of analyzing source code for errors, warnings, and formatting issues before deployment.

Example Checks:

- Unused variables
 - Missing imports
 - Invalid React practices
 - Formatting inconsistencies
-

Package Scripts

A custom lint script was added to package.json.

Example:

```
"scripts": {  
  "dev": "vite",  
  "build": "vite build",  
  "lint": "eslint src --ext js,jsx"  
}
```

This allows linting to be executed using:

```
npm run lint
```

Implementation

ESLint dependencies were configured within the React project.

A lint script was added to package.json.

The project source files were analyzed using:

```
npm run lint
```

ESLint successfully scanned the project and reported code-quality issues related to React configuration and prop validation.

This verified that the linting system was properly integrated and functioning as expected.

Linting Workflow

Developer Writes Code

↓

```
npm run lint
```

↓

ESLint Scans Source Files

↓

Issues Detected

↓

Developer Fixes Issues

↓

Code Quality Improved

Task 7.2 Output Screenshots

```

frontent > {} package-lockjson > ...
1 {
2   "name": "frontent",
3   "version": "0.0.0",
4   "lockfileVersion": 3,
5   "requires": true,
6   "packages": {
7     "": {
8       "name": "frontent",
9       "version": "0.0.0",
10      "dependencies": {
11        "dotenv": "^17.4.2",
12        "react": "^19.2.6",
13        "react-dom": "^19.2.6"
14      },
15      "devDependencies": {
16        "@eslint/js": "^9.39.4",
17        "@types/react": "^19.2.14",
18        "@types/react-dom": "^19.2.3",
19        "@vitejs/plugin-react": "^6.0.1",
20        "eslint": "^9.39.4",
21        "eslint-plugin-react": "^7.37.5",
22        "eslint-plugin-react-hooks": "^7.1.1",
23        "eslint-plugin-react-refresh": "^0.5.2",
24        "globals": "^17.6.0",
25        "vite": "^8.0.12"
26      },
27    },
28    "node_modules/@babel/core": {

```

```

frontent > {} eslint.config.js > ...
1 import js from "@eslint/js";
2 import globals from "globals";
3 import pluginReact from "eslint-plugin-react";
4 import { defineConfig } from "eslint/config";
5
6 export default defineConfig([
7   { files: ["**/*.js,mjs,cjs,jsx"], plugins: { js }, extends: ["js/recommended"], languageOptions: { globals: globals.browser } },
8   pluginReact.configs.flat.recommended,
9 ]);
10

```

```

C:\Users\yashm\Downloads\insta dot analytics internship assignments\Frontent\src\App.jsx
13:9   error  'React' must be in scope when using JSX  react/react-in-jsx-scope
15:13  error  'React' must be in scope when using JSX  react/react-in-jsx-scope
20:21  error  'React' must be in scope when using JSX  react/react-in-jsx-scope

C:\Users\yashm\Downloads\insta dot analytics internship assignments\Frontent\src\components\StudentCard.jsx
1:24   error  'name' is missing in props validation    react/prop-types
1:30   error  'age' is missing in props validation     react/prop-types
5:9    error  'React' must be in scope when using JSX  react/react-in-jsx-scope
13:13  error  'React' must be in scope when using JSX  react/react-in-jsx-scope
15:13  error  'React' must be in scope when using JSX  react/react-in-jsx-scope

C:\Users\yashm\Downloads\insta dot analytics internship assignments\Frontent\src\context\StudentContext.jsx
5:28   error  'children' is missing in props validation react/prop-types
28:9   error  'React' must be in scope when using JSX  react/react-in-jsx-scope

10 problems (10 errors, 0 warnings)
C:\Users\yashm\Downloads\insta dot analytics internship assignments\Frontent>

```

Task 7.2 Source Code

package.json

```

"scripts": {
  "dev": "vite",
  "build": "vite build",
  "lint": "eslint src --ext js,jsx"
}

```

```
}
```

Terminal Command

```
npm run lint
```

Conclusion

Day 7 focused on establishing scalable frontend architecture and improving development workflow quality. React Context API was implemented to standardize global state management across the project, while ESLint was integrated to enforce coding standards and detect potential issues automatically. These enhancements improve maintainability, collaboration, and code quality for future development activities.